Like `password_hash()`, there are multiple HMAC algorithms available:

```
print_r(hash_hmac_algos());
```

```
Output:
Array
(
    [0] => md2
    [1] => md4
    [2] => md5
    [3] => sha1
    [4] => sha224
    ...
```

The full list will vary according to what's installed on your system, but there's likely going to be dozens. The relative merits and differences of hashing algorithms are a pretty specialized topic and not one we will explore here. If you have an interest, then there are ample resources to be found via your favorite search engine.

Encryption

Let's close out this section by looking at how to fully encrypt our data. Remember that hashing is one-way: it is impossible to compute the original data from which a hash is computed. We've seen how to use hashes to verify message (and sender) integrity, but how do we conceal the messages themselves such that if intercepted they are unreadable? This isn't much of a concern if your application is running over HTTPS, because the "S" portion is doing encryption for you: modern HTTPS systems have TLS – transport layer security – embedded, and this protocol ensures that sender and receiver have randomly generated session keys (shared secrets that are discarded after use, and never stored), which are used to encrypt and decrypt all the data flowing over the network while the two systems are connected. This is made possible by public/private key pairs: the client uses the server's public key to encrypt a random session key (generated by the client), and the server decrypts using its private key. Once the shared secret is established, it is used until the client disconnects. All of this happens far away from your PHP code; the HTTPS web server handles it for you, and HTTPS is fairly ubiquitous today, thanks to the efforts of the Electronic Frontier Foundation's https://www.eff.org/https-everywhere campaign. Thank you, EFF!